

CS-1204

Object-Oriented Programming

(3+1)

Week 09

By

Engr. Muhammad Awais

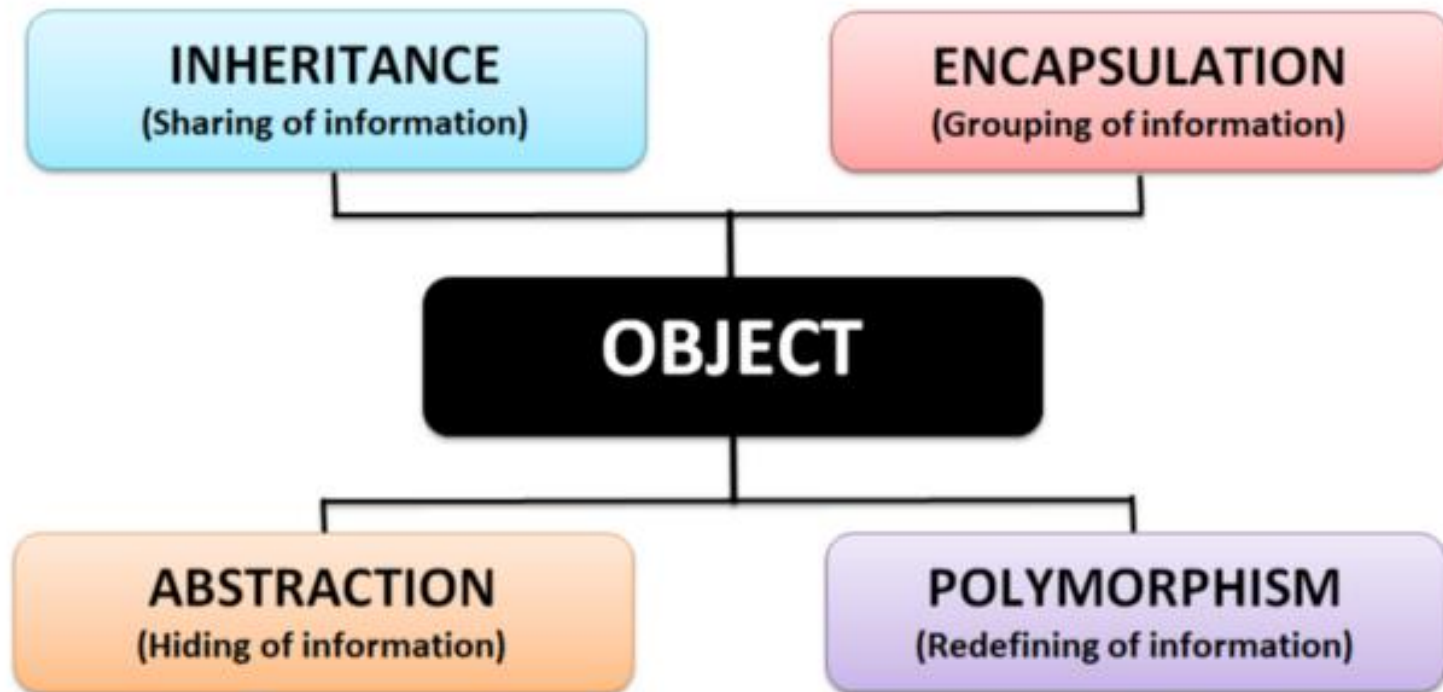
Contents

- Polymorphism in C++
- Function Overloading
- Overriding
- Virtual Functions

Learning Objectives

- Understand the concept of polymorphism
- Differentiate between compile-time and run-time polymorphism
- Understand function overloading and overriding
- Learn the role of virtual functions
- Implement examples and perform hands-on lab tasks

Pillars of OOP



What is Polymorphism?

- Polymorphism means "many forms"
- Same function name behaves differently in different contexts
- Two types in C++:
 - Compile-time polymorphism (Static Binding)
 - Run-time polymorphism (Dynamic Binding)

Compile-Time Polymorphism?

- Achieved using
 - Function Overloading
 - Operator Overloading
- Resolved at compile time – static binding
- Example: **Calculator with multiple add() functions**

Example : Compile-Time Polymorphism

```
#include <iostream>
using namespace std;

class Calculator {
public:
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
};

int main() {
    Calculator calc;
    cout << calc.add(2, 3) << endl;           // 5
    cout << calc.add(2.5, 3.5) << endl;       // 6.0
    cout << calc.add(1, 2, 3) << endl;       // 6
    return 0;
}
```

Run-Time Polymorphism?

- Achieved using
 - Function Overriding and Virtual Functions
- Resolved at run time
- Based on object type (dynamic binding)

Function Overriding?

- Same function name and signature in base and derived class
- Involves inheritance
- Used to provide specific implementation in derived class.

Function Overriding Example Without Virtual Functions

```
#include <iostream>
using namespace std;

class Animal {
public:
    void makeSound() {
        cout << "Animal sound" << endl;
    }
};

class Dog : public Animal {
public:
    void makeSound() {
        cout << "Dog barks" << endl;
    }
};

int main() {
    Animal* a = new Dog();
    a->makeSound(); // Outputs: Animal sound (not Dog barks)
    return 0;
}
```

Function Overriding Example With Virtual Functions

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void makeSound() {
        cout << "Animal sound" << endl;
    }
};

class Dog : public Animal {
public:
    void makeSound() override {
        cout << "Dog barks" << endl;
    }
};

int main() {
    Animal* a = new Dog();
    a->makeSound(); // Outputs: Dog barks
    return 0;
}
```

Practice Example

Shape Area

- Create base class Shape with virtual double area()
- Derive Rectangle and Circle
- Override area() to compute actual values
- Use Shape* pointer to access different shapes.

References

<https://www.geeksforgeeks.org/cpp/cpp-polymorphism/>
https://www.w3schools.com/cpp/cpp_polymorphism.asp